

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

CS 401/MCS 401 Lecture 19
Computer Algorithms I
Jan Verschelde, 31 July 2026

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

the independent set problem

Given a graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set if for all $u, v \in S$: $(u, v) \notin E$. Given a graph G and some number k , does G contain an independent set of size at least k ?

What is good about the independent set problem?

If we have an independent set of size k in a graph, then with the independent set of size k as a given, **verification** that the graph has indeed an independent set of size k is easy.

the 3-satisfiability problem

The 3-satisfiability problem, or 3-SAT for short is

Given $X = \{ x_1, x_2, \dots, x_n \}$ a set of Boolean variables, and $C_1, C_2, \dots, C_k$, a set of clauses over X , where each clause C_i has exactly three terms, does there exist a satisfying truth assignment?

What is good about the 3-SAT problem?

If we have a truth assignment for a 3-SAT problem, then with the assignment of the variables as a given, **verification** that the 3-SAT problem is satisfiable is easy.

computing and verifying a solution

The point is that there is a difference between

- computing a solution to a difficult problem, and
- verifying whether a given output is indeed a solution.

If we can verify efficiently if an output is indeed a solution, then the following algorithm applies:

- 1 Propose an output, by guessing, or generating random numbers.
- 2 If the output is a solution, then we are done.
- 3 If the output is not a solution, then go to step 1.

We may get lucky, but we cannot even hope to terminate.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

on problems and algorithms

On problems:

- The input to our problem is encoded by a finite binary string s . The length of a string s is denoted by $|s|$.
- A decision problem X is identified with the set of strings on which the answer is "yes".

On algorithms:

- An algorithm A takes on input a string s and returns "yes" or "no".
A **solves** the problem X if for all strings s :
$$A(s) = \text{"yes"} \Leftrightarrow s \in X.$$
- Algorithm A has polynomial running time if there is a polynomial function p so that for every input string s ,
 A terminates on s in at most $O(p(|s|))$ steps.

the set of problems that can be solved efficiently

An algorithm is efficient if its running time is bounded by a polynomial function in the input size.

Definition (1)

The set $\mathcal{P}$ is the set of all problems X for which there exists an algorithm A with a polynomial running time that solves X .

All problems in the set $\mathcal{P}$ can be solved efficiently.

efficient certification

To check a solution for a problem X , we need the input string s and a separate certificate string t which contains evidence that s is a "yes" instance of X .

Definition (2)

Algorithm B is an *efficient certifier for problem X* if

- B runs in polynomial time on the inputs s and t , and
- there is a polynomial function p so that for every string s , we have $s \in X$ if and only if there exists a string t such that $|t| \leq p(|s|)$ and $B(s, t) = \text{"yes"}$.

An efficient certifier

- evaluates proposed proofs if they are not too long,
- could be used in a brute force algorithm: try all inputs.

the class of problems $\mathcal{NP}$

A certifier verifies an output of a problem.

The running time of an efficient certifier is bounded by a polynomial in the input size.

Definition (3)

The set $\mathcal{NP}$ collects all problems for which an efficient certifier exists.

The set $\mathcal{P}$ collects all problems for which an efficient algorithm exists.

an inclusion theorem

Theorem (4)

$$\mathcal{P} \subseteq \mathcal{NP}$$

Proof. Consider a problem $X \in \mathcal{P}$

So there is polynomial-time algorithm A to solve X .

To show that $X \in \mathcal{NP}$, we need to find an efficient certifier for X .

For inputs s and t , the certifier algorithm B returns the value of $A(s)$.

- Why is B a efficient? A runs in a polynomial time.
- Why is B a certifier? Recall that X is decision problem.

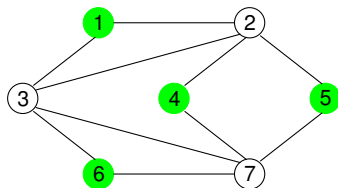
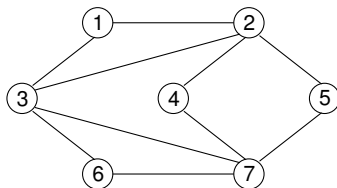
If $s \in X$, then for every t of length at most $p(|s|)$,

$$B(s, t) := \text{"yes"} \text{ if } (A(s) = \text{"yes"}), \text{ else "no"}.$$

Q.E.D.

an efficient certifier for the independent set problem

Given a graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set if for all $u, v \in S$: $(u, v) \notin E$. Given a graph G and some number k , does G contain an independent set of size at least k ?

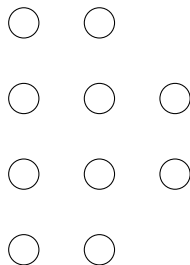


The certificate t is the set S of at least k vertices.

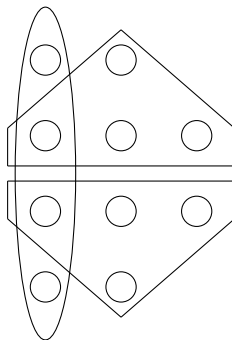
The certifier checks that, for all pairs $u, v \in S$: $(u, v) \notin E$.

an efficient certifier for the set cover problem

Given are $U = \{ 1, 2, \dots, n \}$, a collection $S_i \subseteq U, i = 1, 2, \dots, m$, and some number k . Is there an index set $I = \{ i_1, i_2, \dots, i_k \} : \bigcup_{j \in I} S_j = U$?



a collection



a set cover

The certificate t is the sequence of k subsets of U .

The certifier checks that the union of the subsets equals U .

an efficient certifier for the 3-satisfiability problem

The conjunction of clauses

$$(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

has a satisfying truth assignment $\nu(x) = 1$, $\nu(y) = 0$, $\nu(z) = 0$.

The certificate t is an assignment ν to the Boolean variables.

The certifier evaluates the clauses with respect to the assignment ν .

efficient certification of set packing

The set packing problem:

Given a set U of n elements,
a collection $S_1, S_2, \dots, S_m$ of subsets of U , and
some number k ,
does there exist a collection of at least k of these sets
so that no two of them intersect?

Exercise 1:

Formulate an efficient certifier for the set packing problem.

the question

Is there a problem in $\mathcal{NP}$ that does not belong to $\mathcal{P}$?

Does $\mathcal{P} = \mathcal{NP}$?

The general belief is $\mathcal{P} \neq \mathcal{NP}$.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- **NP-Complete Problems**

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

NP-complete problems

What are the hardest problems in $\mathcal{NP}$?

Definition (5)

Problem X is an *NP-complete problem* if

- 1 $X \in \mathcal{NP}$, and
- 2 for all $Y \in \mathcal{NP}$: $Y \leq_P X$.

Every problem in $\mathcal{NP}$ can be reduced to an NP-complete problem.

Theorem (6)

Suppose X is an NP-complete problem.

Then X is solvable in polynomial time if and only if $\mathcal{P} = \mathcal{NP}$.

proof by applying definitions

Theorem (6)

Suppose X is an NP-complete problem.

Then X is solvable in polynomial time if and only if $\mathcal{P} = \mathcal{NP}$.

Proof. If and only if means $\Rightarrow$ and $\Leftarrow$.

$\Leftarrow$ If $\mathcal{P} = \mathcal{NP}$,

then X can be solved in polynomial time as $X \in \mathcal{NP}$.

$\Rightarrow$ Suppose X can be solved in polynomial time.

For all $Y \in \mathcal{NP}$: $Y \leq_P X$.

If $Y \leq_P X$ and X can be solved in polynomial time,
then Y can be solved in polynomial time, and thus $Y \in \mathcal{P}$.

So, $\mathcal{NP} \subseteq \mathcal{P}$.

By Theorem (4), $\mathcal{P} \subseteq \mathcal{NP}$, so $\mathcal{P} = \mathcal{NP}$.

Q.E.D.

why NP-complete problems are important

Let us consider an important consequence of the theorem.

Theorem (6)

Suppose X is an NP-complete problem.

Then X is solvable in polynomial time if and only if $\mathcal{P} = \mathcal{NP}$.

Corollary (7)

If there is any problem in $\mathcal{NP}$ that cannot be solved in polynomial time, then no NP-complete problem can be solved in polynomial time.

encoding a problem in $\mathcal{NP}$

Do NP-complete problems exist?

Every problem in $\mathcal{NP}$ can be reduced to an NP-complete problem.

Could there be two incomparable problems Y and Z so that there is no $X \in \mathcal{NP}$ so that $Y \leq_P X$ and $Z \leq_P X$?

To prove a problem is NP-complete, we must show how it could encode *any* problem in $\mathcal{NP}$.

This is harder than reformulating specific problems.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

CS 401/MCS 401 Lecture 19
Computer Algorithms I
Jan Verschelde, 31 July 2026

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

motivation

We defined the following.

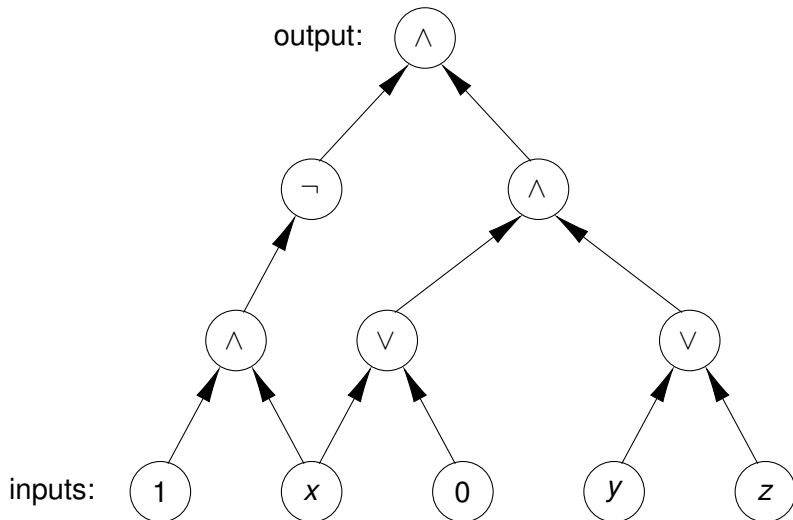
- 1 An algorithm is efficient if its worst case running time is bounded by a polynomial in the input size.
- 2 $\mathcal{P}$ collects all problems for which an efficient algorithm exists.
- 3 A certifier verifies an output of a problem.
- 4 $\mathcal{NP}$ collects all problems for which an efficient certifier exists.

We proved the following theorem.

*If X is an NP-complete problem,
then X is solvable in polynomial time if and only if $\mathcal{P} = \mathcal{NP}$.*

To prove a problem is NP-complete,
we must show how it could encode *any* problem in $\mathcal{NP}$.

a circuit: $\neg(1 \wedge x) \wedge ((x \vee 0) \wedge (y \vee z))$



definition of a circuit

A *circuit* K is a labeled, directed acyclic graph:

- The *sources* in K are nodes with no incoming edges.
The sources are labeled either with a constant 0 or 1,
or with the name of a distinct variable.
Nodes labeled with the name of a variable are *inputs* to the circuit.
- Internal nodes are labeled with a Boolean operation,
which is either the negation $\neg$ (not), the disjunction $\vee$ (or),
or the conjunction $\wedge$ (and).
Nodes labeled with $\neg$ have one incoming edge,
nodes labeled with $\vee$ or $\wedge$ have two incoming edges.
- The *output* of the circuit is one single sink, with no outgoing edges.
The output represents what the circuit computes.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- **the circuit satisfiability problem**
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

the circuit satisfiability problem

Assignments to the inputs of a circuit that results in the output one is called a satisfying assignment.

Definition (8)

A circuit K is *satisfiable* if there exist values for the inputs of K so that the output of K equals one.

The Circuit Satisfiability Problem is then

Given a circuit K , decide whether K is satisfiable.

Theorem (9)

Circuit Satisfiability is NP-complete.

an example

Consider the following problem:

Given a graph $G = (V, E)$, does it contain a 2-node independent set?

Let $n = \#V$.

The input s can be represented by $\binom{n}{2}$ bits:

for each pair of nodes u, v , there will be a constant

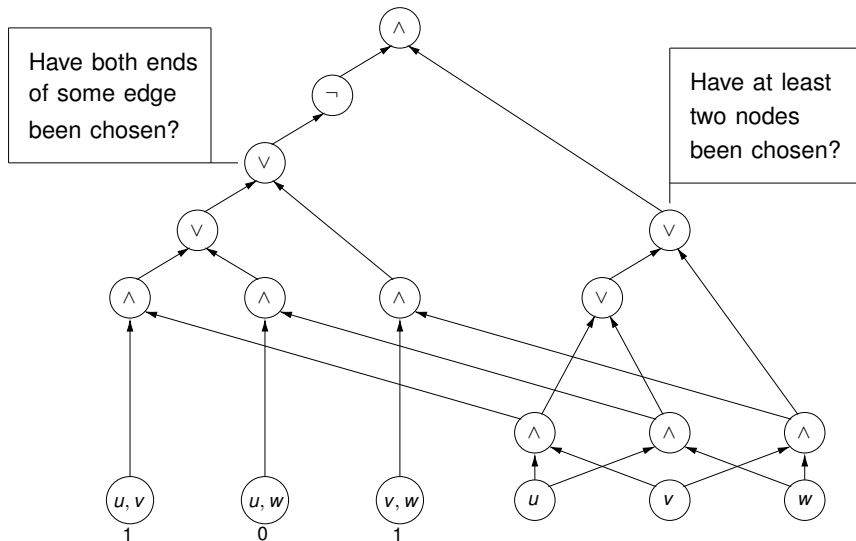
- 1 if $(u, v) \in E$, or
- 0 if $(u, v) \notin E$.

The certificate t contains n bits:

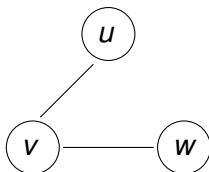
- 1 if the vertex belongs to the independent set, or
- 0 otherwise.

The next slide shows a circuit for a certifier in a 3-node graph.

a circuit for a certifier for a 2-node independent set



an expression for a 2-node independent set



Constants are $e_{(u,v)} = 1$, $e_{(u,w)} = 0$, $e_{(v,w)} = 1$, with inputs u , v , and w .

Choose at least two nodes:

$$((u \wedge v) \vee (u \wedge w) \vee (v \wedge w))$$

and choose both ends of some edge:

$$\neg((e_{(u,v)} \wedge (u \wedge v)) \vee (e_{(u,w)} \wedge (u \wedge w)) \vee (e_{(v,w)} \wedge (v \wedge w))),$$

which is satisfied by $u = 1$, $v = 0$, $w = 1$.

the main theorem

Theorem (10)

Circuit Satisfiability is NP-complete.

We need to show that for an arbitrary problem X in $\mathcal{NP}$:

$$X \leq_P \text{Circuit Satisfiability.}$$

Steps in the proof:

- 1 Algorithms can be written as circuits.
- 2 Any problem $X \in \mathcal{NP}$, has an efficient certifier algorithm B .
- 3 Call the black box solver for Circuit Satisfiability.

step 1: algorithms can be formulated as circuits

- Any algorithm for a decision problem takes on input a sequence of n bits and returns either 1 (for "yes") or 0 (for "no").
- The input sequence of bits for the algorithm are the sources for a circuit.
The output of the circuit is the outcome of the algorithm.
- If the algorithm takes a number of steps that is polynomial in n , then the circuit has polynomial size.
- All computational steps on a real computer are implemented to run on operations with $\wedge$, $\vee$, and $\neg$ gates.

step 2: the certifier algorithm as circuit

Any problem $X \in \mathcal{NP}$ has an efficient certifier algorithm B , with two inputs:

- 1 a string s for the input of the problem,
- 2 a string t as a certificate of the output of the problem.

For $X \leq_P$ Circuit Satisfiability,
for an input s , we want to decide $s \in X$
using a black box that can solve instances of Circuit Satisfiability.

step 3: the black box solver for Circuit Satisfiability

We call the black box solver for Circuit Satisfiability as follows:

- We view $B(s, t)$ as an algorithm on $n + p(n)$ bits, where $n = |s|$, $p(n) = |t|$.
- The algorithm B can be formulated as a circuit K with $n + p(n)$ sources:
 - 1 the first n bits are the constants for the source nodes,
 - 2 the next $p(n)$ bits are the nodes labeled with variable names.

Now we observe that $s \in X$

- if and only if there is a way to set the input bits to K so that the circuit produces the output of 1, or equivalently,
- if and only if K is satisfiable.

Thus, $X \leq_P$ Circuit Satisfiability. Since X is an arbitrary problem in $\mathcal{NP}$, we have that Circuit Satisfiability is NP-complete.

Q.E.D.

transitivity

Once we have one NP-complete problem, we can find many more.

Proposition (11)

Let Y be an NP-complete problem.

If $X \in \mathcal{NP}$ with $Y \leq_P X$, then X is NP-complete.

Proof. As $X \in \mathcal{NP}$, we need to verify the second part of the definition of an NP-Complete problem: for all $Z \in \mathcal{NP}$: $Z \leq_P X$.

By the NP-completeness of Y , we have $Z \leq_P Y$.

Given is that $Y \leq_P X$.

By the transitivity of $\leq_P$, we have then that $Z \leq_P X$.

Thus X is NP-complete.

Q.E.D.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

the 3-satisfiability problem

The 3-satisfiability problem, or 3-SAT for short is

Given $X = \{ x_1, x_2, \dots, x_n \}$ a set of Boolean variables, and $C_1, C_2, \dots, C_k$, a set of clauses over X , where each clause C_i has exactly three terms, does there exist a satisfying truth assignment?

The example

$$(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

is satisfied by $x = 1$, $\bar{y} = 1$, and $\bar{z} = 1$.

3-satisfiability

Theorem (12)

3-Satisfiability is NP-complete

Proof. We have an efficient certifier algorithm for 3-SAT,
so 3-satisfiability is in $\mathcal{NP}$.

NP-completeness will be proven by reduction to Circuit Satisfiability.

Given an arbitrary instance of Circuit Satisfiability,

- 1 we construct an equivalent instance of SAT
where every clause has at most three variables,
- 2 then convert the SAT instance to an instance of 3-SAT,
where every clause has exactly 3 variables.

from a circuit to an instance of SAT

Given a circuit K , apply the following rules:

- ① To every node v in K , associate the variable x_v .
- ② At the internal nodes of K , we have gates of type $\neg$, $\vee$, and $\wedge$.
 - ① $\neg$ at node v with entering node u :
add clauses $(x_v \vee x_u), (\bar{x}_v \vee \bar{x}_u)$.

Exercise 2: verify that the conjunction of clauses
gives $x_v = \bar{x}_u$.

from a circuit to an instance of SAT, continued

Given a circuit K , apply the following rules:

- ① To every node v in K , associate the variable x_v .
- ② At the internal nodes of K , we have gates of type $\neg$, $\vee$, and $\wedge$.
 - ② $\vee$ at node v with entering nodes u and w :
add clauses $(x_v \vee \bar{x}_u)$, $(x_v \vee \bar{x}_w)$, $(\bar{x}_v \vee x_u \vee x_w)$

Exercise 3: verify that the conjunction of clauses gives $x_v = x_u \vee x_w$.

- ③ $\wedge$ at node v with entering nodes u and w :
add clauses $(\bar{x}_v \vee x_u)$, $(\bar{x}_v \vee x_w)$, $(x_v \vee \bar{x}_u \vee \bar{x}_w)$.

Exercise 4: verify that the conjunction of clauses gives $x_v = x_u \wedge x_w$.

inputs and output of the circuit

For a source v which has been labeled with a constant value,

- add the clause x_v , if the constant is 1, or
- add the clause $\bar{x}_v$, if the constant is 0.

To the output of the circuit corresponds the node o :

- add the clause x_o , which requires that o takes the value 1.

Lemma (13)

The constructed SAT instance is equivalent to the given circuit.

The proof goes in both directions: if the circuit is satisfiable, then the SAT instance has a satisfying truth statement.

In the other direction: if the SAT instance has a satisfying truth statement, then the circuit is satisfiable.

another exercise

Exercise 5: For the circuit

$$\neg(1 \wedge x) \wedge ((x \vee 0) \wedge (y \vee z)),$$

shown on slide 26, construct an instance of SAT.

construction of an instance of 3-SAT

All clauses in the construction have at most three terms.
Now we have to construct an instance of 3-SAT,
where every clause has exactly three terms.

We add four variables z_1, z_2, z_3, z_4 and add the clauses

$$(\bar{z}_i \vee z_3 \vee z_4), (\bar{z}_i \vee \bar{z}_3 \vee z_4), \quad i = 1, 2$$

and

$$(\bar{z}_i \vee z_3 \vee \bar{z}_4), (\bar{z}_i \vee \bar{z}_3 \vee \bar{z}_4), \quad i = 1, 2.$$

The conjunction of these clauses forces $z_1 = 0$ and $z_2 = 0$.

- 1 Replace clauses with one term t by $(t \vee z_1 \vee z_2)$.
- 2 Replace clauses with two terms t and t' by $(t \vee t' \vee z_1)$.

So we have Circuit Satisfiability $\leq_P$ 3-SAT.

Q.E.D.

more NP-complete problems

Corollary (14)

*All of the following problems are NP-complete:
Independent Set, Set Packing, Vertex Cover, and Set Cover.*

Proof. We had the sequence of reductions:

$$3\text{-SAT} \leq_P \text{Independent Set} \leq_P \text{Vertex Cover} \leq_P \text{Set Cover}.$$

Each of these problems has an efficient certifier, belongs to $\mathcal{NP}$, and that 3-SAT can be reduced to it.

Q.E.D.

the strongly independent set problem

Exercise 6:

Given is a graph $G = (V, E)$, and an integer k .

A subset I of V is strongly independent if for any $u, v \in I$,

- $(u, v) \notin E$, and
- there is no w such that $(u, w) \in E$ and $(w, v) \in E$.

The strongly independent set problem asks if G contains a strongly independent set of size k .

Prove that the strongly independent set problem is NP-complete.

strategy for proving a new problem to be NP-complete

To prove that a new problem X is NP-complete:

- 1 Prove that $X \in \mathcal{NP}$.
- 2 Choose a problem Y known to be NP-complete.
- 3 Prove that $Y \leq_P X$.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

CS 401/MCS 401 Lecture 19
Computer Algorithms I
Jan Verschelde, 31 July 2026

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

bipartite matching

We studied the Bipartite Matching Problem.

Given is a bipartite graph $G = (V, E)$, $V = X \cup Y$, $X \cap Y = \emptyset$.

A matching is a subset M of E such that every vertex occurs at most once in M .

A matching M is perfect if all vertices occur in M .

The bipartite matching problem asks

Does a given bipartite graph have a perfect matching?

The Ford-Fulkerson algorithm solves this problem efficiently.

the 3-dimensional matching problem

Given are the following:

- Three sets X , Y , and Z :
 - 1 each set has the same size $n = \#X = \#Y = \#Z$, and
 - 2 the sets are disjoint: $X \cap Y = \emptyset$, $X \cap Z = \emptyset$, $Y \cap Z = \emptyset$.
- A set T of ordered triples $T \subseteq X \times Y \times Z$.

The 3-dimensional matching problem asks

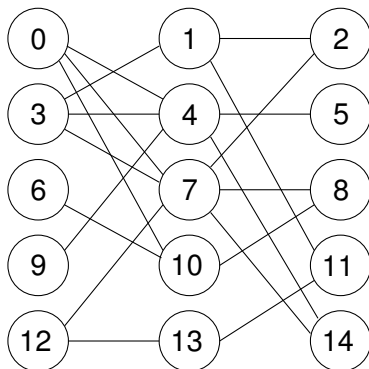
Is there a set of n triples in T so each element of $X \cup Y \cup Z$ is contained *exactly once* in one of these triples?

Such a set of triples is *a perfect 3-dimensional matching*.

Example: $X = \{\text{instructors}\}$, $Y = \{\text{courses}\}$, $Z = \{\text{times}\}$.

an example of a 3-dimensional matching problem

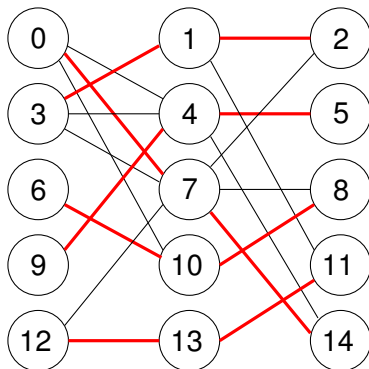
$$X = \{ 0, 3, 6, 9, 12 \}, Y = \{ 1, 4, 7, 10, 13 \}, Z = \{ 2, 5, 8, 11, 14 \}.$$



T is defined by the edges of the graph.

a solution to the example

$X = \{ 0, 3, 6, 9, 12 \}$, $Y = \{ 1, 4, 7, 10, 13 \}$, $Z = \{ 2, 5, 8, 11, 14 \}$.



A solution: $\{(0, 7, 14), (3, 1, 2), (6, 10, 8), (9, 4, 5), (12, 13, 11)\}$.

solve the 3-dimensional matching problem

The 3-dimensional matching problem is important.

Exercise 7: Solve the 3-dimensional matching problem by giving an algorithm to find a perfect matching in a tripartite graph.

What is the worst case running time of your algorithm?

the set cover problem

The set cover problem is a decision problem:

Given are $U = \{ 1, 2, \dots, n \}$,
a collection $S_i \subseteq U, i = 1, 2, \dots, m$,
and some number k .

Is there an index set $I = \{ i_1, i_2, \dots, i_k \} : \bigcup_{j \in I} S_j = U$?

We proved that $3\text{-SAT} \leq_P \text{Set Cover}$. Set Cover is NP-Complete.

The 3-dimensional matching problem is a special case of set cover:

- $U = X \cup Y \cup Z$.
- We cover at most n sets from the given collection of triples.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- **NP-completeness**
- reducing 3-SAT to 3-dimensional matching

strategy to show a new problem is NP-complete

Given a new problem X , to show X is NP-complete:

- 1 Prove that $X \in \mathcal{NP}$.

This consists in exhibiting an efficient certifier algorithm.

- 2 Choose a problem Y that is known to be NP-complete.

- 3 Prove that $Y \leq_P X$.

The reformulation $Y \leq_P X$ implies that

- ▶ we can solve Y by calling the black box solver for X , and
- ▶ a polynomial number of elementary computational steps.

As the 3-dimensional matching problem is a special case of Set Cover, proving 3-Dimensional Matching $\leq_P$ Set Cover is not hard.

However, to show 3-Dimensional Matching is NP-complete, we must show the other direction, that a known NP-complete problem can be reduced to the 3-Dimensional Matching problem.

3-dimensional matching is in $\mathcal{NP}$

Lemma (1)

The 3-Dimensional Matching Problem is in $\mathcal{NP}$.

Proof. A certificate for the 3-dimensional matching problem is a sequence of triples. The certifier algorithm takes on input this certificate and the input to the problem: the sets X , Y , Z , and the triples T . Let $n = \#X = \#Y = \#Z$ and $U = X \cup Y \cup Z$.

To verify the result:

- 1 We check that every point in U occurs in the certificate.
This can be done with one scan through U and $3n$ steps, using a Boolean array of size $3n$.
- 2 We check that every triple in the certificate belongs to T .
If $m = \#T$, then the check for one triple is $O(m)$.
For all n triples, the check runs in $O(nm)$ time.

By the polynomial-time running time, the problem is in $\mathcal{NP}$.

Q.E.D.

the 3-satisfiability problem is NP-complete

The 3-satisfiability problem, or 3-SAT for short is

Given $X = \{ x_1, x_2, \dots, x_n \}$ a set of Boolean variables, and $C_1, C_2, \dots, C_k$, a set of clauses over X , where each clause C_i has exactly three terms, does there exist a satisfying truth assignment?

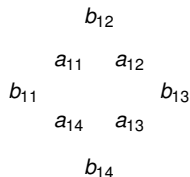
The example

$$(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

is satisfied by $x = 1$, $\bar{y} = 1$, and $\bar{z} = 1$.

triples and gadgets

$$T = \{(a_{11}, a_{12}, b_{12}), (a_{12}, a_{13}, b_{13}), (a_{13}, a_{14}, b_{14}), (a_{14}, a_{11}, b_{11})\}.$$



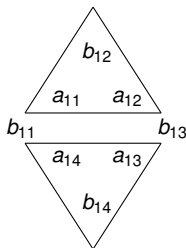
- 1 $a_{11}, a_{12}, a_{13}, a_{14}$ are the *core* of the gadget;
- 2 $b_{11}, b_{12}, b_{13}, b_{14}$ are the *tips* of the gadget.

A triple is even or odd if j of b_{ij} is even or odd.

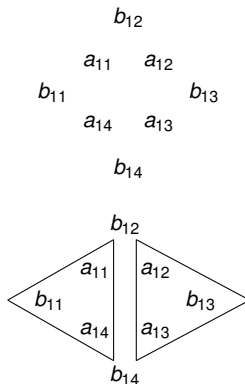
To cover the core, use either all even or all odd triples.

even/odd triples and 0/1 assignments

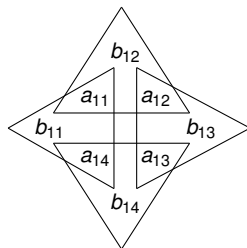
$$T = \{(a_{11}, a_{12}, b_{12}), (a_{12}, a_{13}, b_{13}), (a_{13}, a_{14}, b_{14}), (a_{14}, a_{11}, b_{11})\}.$$



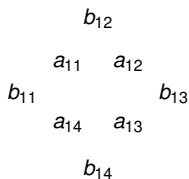
$$x_1 = 0$$



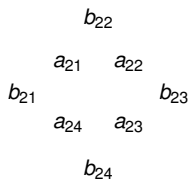
$$x_1 = 1$$



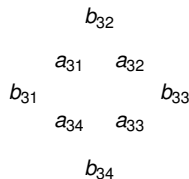
assigning to three variables in a clause



$$x_1 = 0$$

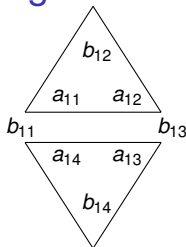


$$x_2 = 1$$

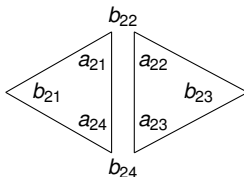


$$x_3 = 0$$

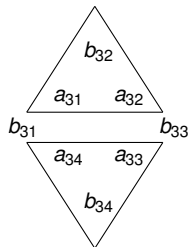
making a truth table



$$x_1 = 0$$



$$x_2 = 1$$



$$x_3 = 0$$

111: $(a_{11}, a_{14}, b_{11}), (a_{12}, a_{13}, b_{13}), (a_{21}, a_{24}, b_{21}), (a_{22}, a_{23}, b_{23}), (a_{31}, a_{34}, b_{31}), (a_{32}, a_{33}, b_{33})$

110: $(a_{11}, a_{14}, b_{11}), (a_{12}, a_{13}, b_{13}), (a_{21}, a_{24}, b_{21}), (a_{22}, a_{23}, b_{23}), (a_{31}, a_{32}, b_{32}), (a_{33}, a_{34}, b_{34})$

101: $(a_{11}, a_{14}, b_{11}), (a_{12}, a_{13}, b_{13}), (a_{21}, a_{22}, b_{22}), (a_{23}, a_{24}, b_{24}), (a_{31}, a_{34}, b_{31}), (a_{32}, a_{33}, b_{33})$

100: $(a_{11}, a_{14}, b_{11}), (a_{12}, a_{13}, b_{13}), (a_{21}, a_{22}, b_{22}), (a_{23}, a_{24}, b_{24}), (a_{31}, a_{32}, b_{32}), (a_{33}, a_{34}, b_{34})$

011: $(a_{11}, a_{12}, b_{12}), (a_{13}, a_{14}, b_{14}), (a_{21}, a_{24}, b_{21}), (a_{22}, a_{23}, b_{23}), (a_{31}, a_{34}, b_{31}), (a_{32}, a_{33}, b_{33})$

010: $(a_{11}, a_{12}, b_{12}), (a_{13}, a_{14}, b_{14}), (a_{21}, a_{24}, b_{21}), (a_{22}, a_{23}, b_{23}), (a_{31}, a_{32}, b_{32}), (a_{33}, a_{34}, b_{34})$

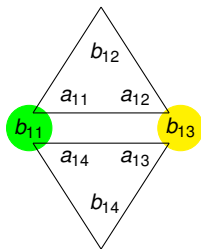
001: $(a_{11}, a_{12}, b_{12}), (a_{13}, a_{14}, b_{14}), (a_{21}, a_{22}, b_{22}), (a_{23}, a_{24}, b_{24}), (a_{31}, a_{34}, b_{31}), (a_{32}, a_{33}, b_{33})$

000: $(a_{11}, a_{12}, b_{12}), (a_{13}, a_{14}, b_{14}), (a_{21}, a_{22}, b_{22}), (a_{23}, a_{24}, b_{24}), (a_{31}, a_{32}, b_{32}), (a_{33}, a_{34}, b_{34})$

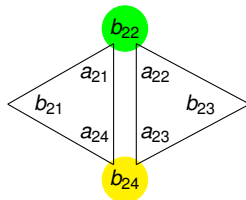
a clause in three variables

The clause $\bar{x}_1 \vee x_2 \vee \bar{x}_3$ is represented by the three triples:
 (p, p', b_{11}) , (p, p', b_{22}) , and (p, p', b_{31}) .

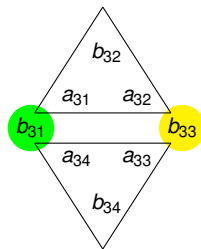
For a perfect 3-dimensional matching corresponding to a satisfying assignment, add (q_1, q'_1, b_{22}) , (q_2, q'_2, b_{31}) , (q_3, q'_3, b_{13}) , (q_4, q'_4, b_{24}) , (q_5, q'_5, b_{33}) . In total, we have $8 = 2^3$ triples, for 3 variables.



$$x_1 = 0$$

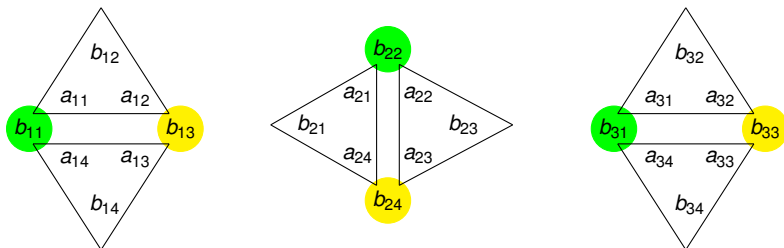


$$x_2 = 1$$



$$x_3 = 0$$

an instance of 3-dimensional matching



- triples for the assignments of x_1 , x_2 , and x_3 :
 (a_{11}, a_{12}, b_{12}) , (a_{12}, a_{13}, b_{13}) , (a_{13}, a_{14}, b_{14}) , (a_{14}, a_{11}, b_{11}) ,
 (a_{21}, a_{22}, b_{22}) , (a_{22}, a_{23}, b_{23}) , (a_{23}, a_{24}, b_{24}) , (a_{24}, a_{21}, b_{21}) ,
 (a_{31}, a_{32}, b_{32}) , (a_{32}, a_{33}, b_{33}) , (a_{33}, a_{34}, b_{34}) , (a_{34}, a_{31}, b_{31}) .
- triples for the clause: (p, p', b_{11}) , (p, p', b_{22}) , (p, p', b_{31}) .
- triples to make the matching perfect: (q_1, q'_1, b_{22}) , (q_2, q'_2, b_{31}) ,
 (q_3, q'_3, b_{13}) , (q_4, q'_4, b_{24}) , (q_5, q'_5, b_{33}) .

a 3-dimensional matching problem

- triples for the assignments of x_1 , x_2 , and x_3 :
 $(a_{11}, a_{12}, b_{12}), (a_{12}, a_{13}, b_{13}), (a_{13}, a_{14}, b_{14}), (a_{14}, a_{11}, b_{11}),$
 $(a_{21}, a_{22}, b_{22}), (a_{22}, a_{23}, b_{23}), (a_{23}, a_{24}, b_{24}), (a_{24}, a_{21}, b_{21}),$
 $(a_{31}, a_{32}, b_{32}), (a_{32}, a_{33}, b_{33}), (a_{33}, a_{34}, b_{34}), (a_{34}, a_{31}, b_{31}).$
- triples for the clause: $(p, p', b_{11}), (p, p', b_{22}), (p, p', b_{31}).$
- triples to make the matching perfect: $(q_1, q'_1, b_{22}), (q_2, q'_2, b_{31}),$
 $(q_3, q'_3, b_{13}), (q_4, q'_4, b_{24}), (q_5, q'_5, b_{33}).$

Three sets X , Y , and Z :

- $X = \{a_{11}, a_{13}, a_{21}, a_{23}, a_{31}, a_{33}, p, q_1, q_2, q_3, q_4, q_5\}$
- $Y = \{a_{12}, a_{14}, a_{22}, a_{24}, a_{32}, a_{34}, p', q'_1, q'_2, q'_3, q'_4, q'_5\}$
- $Z = \{b_{11}, b_{12}, b_{13}, b_{14}, b_{21}, b_{22}, b_{23}, b_{24}, b_{31}, b_{32}, b_{33}, b_{34}\}$

We can verify that a perfect 3-dimensional matching corresponds to a satisfying truth assignment for the clause.

Certification and NP-Completeness

1 Definition of $\mathcal{NP}$

- computing and verifying solutions
- the sets $\mathcal{P}$ and $\mathcal{NP}$
- NP-Complete Problems

2 Circuit Satisfiability

- circuits as labeled directed acyclic graphs
- the circuit satisfiability problem
- 3-satisfiability is NP-complete

3 The 3-Dimensional Matching Problem

- a general version of bipartite matching
- NP-completeness
- reducing 3-SAT to 3-dimensional matching

reducing 3-SAT to 3-dimensional matching

Consider an arbitrary instance of the 3-SAT problem:

- 1 k clauses $C_j = c_{j1} \vee c_{j2} \vee c_{j3}$, $j = 1, 2, \dots, k$,
- 2 n variables x_i , $c_{j1}, c_{j2}, c_{j3} \in \{x_1, \bar{x}_1, x_2, \bar{x}_2, \dots, x_n, \bar{x}_n\}$.

For each variable, define the following:

- $A_i = \{a_{i1}, a_{i2}, \dots, a_{i2k}\}$, $B_i = \{b_{i1}, b_{i2}, \dots, b_{i2k}\}$,
- $t_{ij} = (a_{ij}, a_{i,j+1}, b_{ij})$, for $j = 1, 2, \dots, 2k$, where $j + 1 = 1$ if $j = 2k$.

For each clause C_j , add elements p_j , p'_j , and for each term t of C_j :

- if $t = x_i$, add the triple (p_j, p'_j, b_{i2j}) , or otherwise
- if $t = \bar{x}_i$, add the triple (p_j, p'_j, b_{i2j-1}) .

To cover all $2kn$ elements b_{ij} , we have nk covered by assignments to x_i and k covered by the clauses, so we still need $(n - 1)k$ triples:

- add $2(n - 1)$ elements q_i , q'_i ,
- add $(n - 1)k$ triples (q_i, q'_i, b) , for every b not covered by a clause.

To an arbitrary instance of the 3-SAT problem:

- 1 k clauses $C_j = c_{j1} \vee c_{j2} \vee c_{j3}, j = 1, 2, \dots, k,$
- 2 n variables $x_i, c_{j1}, c_{j2}, c_{j3} \in \{x_1, \bar{x}_1, x_2, \bar{x}_2, \dots, x_n, \bar{x}_n\}.$

corresponds an instance of the 3-dimensional matching problem:

- 1 $2nk + 3k + (n - 1)k$ triples T
- 2 X contains all a_{ij} with j odd, the p_j and the q_j elements
- 3 Y contains all a_{ij} with j even, the p'_j and the q'_j elements
- 4 Z contains all b_{ij} elements.

As $2nk + 3k + (n - 1)k$ is a polynomial, we have a polynomial number of computational steps to reduce 3-SAT to 3-dimensional matching.

encoding a 3-SAT instance as 3-dimensional matching

Exercise 8: Encode

$$(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

as a 3-dimensional matching problem.

Demonstrate the interpretation of a truth assignment as a perfect matching in a tripartite graph, and vice versa.

3-SAT is equivalent to 3-dimensional matching

Lemma (2)

The 3-SAT problem is satisfiable if and only if there is a perfect 3-dimensional matching in the set of triples T .

Proof. If and only if means $\Rightarrow$ and $\Leftarrow$.

$\Rightarrow$ If we have a satisfying truth statement, then the triples are chosen corresponding to the 0/1 values for each variable.

For each clause C_j , a triple containing p_j and p'_j is selected.

A choice of the remaining triples which contain q_i and q'_k gives a perfect 3-dimensional matching.

3-SAT is equivalent to 3-dimensional matching

Lemma (2)

The 3-SAT problem is satisfiable if and only if there is a perfect 3-dimensional matching in the set of triples T .

Proof. continued with the $\Leftarrow$ case.

- $\Leftarrow$ If we have a perfect 3-dimensional matching, then the matching contains
- ▶ either all the even triples t_{ij} ,
 - ▶ or all the odd triples t_{ij}

where the indices specify the 0/1 assignment of the variable x_i .

As we have a perfect matching, each p_j and p'_j corresponding to the clause C_j was selected to satisfy C_j .

Q.E.D.

NP-completeness

Theorem (3)

The 3-dimensional matching problem is NP-complete.

Proof. By Lemma (1), the 3-dimensional matching problem is in $\mathcal{NP}$.

An arbitrary 3-SAT problem can be reduced in polynomial time to a 3-dimensional matching problem. In particular, for k clauses in n variables, we construct $2nk + 3k + (n - 1)k$ triples.

By Lemma (2), the reformulated 3-SAT problem can be solved by the 3-dimensional matching problem.

Q.E.D.

partitioning and graph coloring

A graph is k -colorable if k colors suffice to assign colors to all vertices so no two adjacent vertices have the same color.

Given a graph G and a natural number k , does G have a k -coloring?

Theorem (4)

A graph is 2-colorable if and only if it is bipartite.

Reducing 3-SAT to 3-coloring, the following can be proven:

Theorem (5)

3-coloring is NP-complete.

a short summary of this and last lecture

In the last lecture,

- we defined $Y \leq_P X$ as the polynomial-time reducibility to connect hard problems.
- If $Y \leq_P X$ and X can be solved in polynomial time, then Y can be solved in polynomial time.
- Optimization problems reduce to decision problems.
- We looked at packing, covering problems, and satisfiability.

In this lecture, introducing the class $\mathcal{NP}$,

- we identified the interesting hard problems as NP-complete problems.
- Algorithms can be formulated as circuits.
We showed that circuit satisfiability is NP-complete.